

CS217 Assignment 6

Zhige Chen 12413315

*Department of Computer Science and Engineering, Southern University of Science
and Technology, China*

October 18, 2025

Abstract

This project conducts a comprehensive performance evaluation comparing database management systems (PostgreSQL and openGauss) against manual file operations (including the JSON and BEVE format) using an employee database schema. This experiments reveal that PostgreSQL is generally faster than openGauss, and using DBMS can lead to less resource usage and provide significant advantages for complex queries and maintainability.

Contents

1 Project Overview	2
2 Experimental Setup	2
2.1 Environment Configuration	2
2.2 Libraries Used	2
2.3 Measurement Methodology	2
3 Experiment Results	2
3.1 Retrieval Performance Comparison	2
3.1.1 Retrieval Test 1	2
3.1.2 Retrieval Test 2	3
3.1.3 Retrieval Test 3	3
3.1.4 Retrieval Test 4	3
3.1.5 Retrieval Test 5	4
3.2 Update Performance Comparison	4
3.2.1 Update Test 1	4
3.2.2 Update Test 2	4
3.2.3 Update Test 3	5
3.2.4 Update Test 4	5
3.3 Advanced Comparisons: Different File Format	5
4 Analysis and Discussion	6
5 Conclusion	6
A Appendix A: Source Code Repository	7
B Appendix B: Generative AI Usage	7
C Appendix C: Raw Experimental Data	7
C.1 Retrieval Test 1	7
C.2 Retrieval Test 2	8
C.3 Retrieval Test 3	8
C.4 Retrieval Test 4	9
C.5 Retrieval Test 5	9
C.6 Update Test 1	9
C.7 Update Test 2	10
C.8 Update Test 3	10
C.9 Update Test 4	10

D Appendix D: Test Code	10
D.1 Retrieval Test 1	10

1 Project Overview

The **database management systems** (DBMS) have become fundamental to modern data management, but understanding their performance characteristics compared to traditional file operations remains crucial. This projects evaluates PostgreSQL and openGauss against file-based data management using a realistic employee database schema. The benchmarks are implemented in C++.

2 Experimental Setup

2.1 Environment Configuration

- **Hardware:** AMD Ryzen 9 7945HX, 64GB DDR4 4800MHz RAM, SK Hynix PC801 2T SSD
- **PostgreSQL:** Version 18.0.1, default configuration, running in Docker container
- **openGauss:** Version 7.0.0-RC1 build 10d38387, default configuration, running in Docker container
- **Docker:** Version 28.5.1, build e180ab8
- **C++ Compiler:** Clang 21.1.1, target x86_64-w64-windows-gnu
- **Dataset:** Employee data with 3919015 records and 333MB installed size, see the [Github repository](#)

2.2 Libraries Used

- **libpqxx** is used to interact with PostgreSQL and openGauss.
- **Catch2** is used to write unit tests and benchmarks for the experiments.
- **Glaze** is used to read/write JSON and BEVE format files.

2.3 Measurement Methodology

The timing measurements are all conducted using the `ADVANCED_BENCHMARK()` marco of **Catch2** library. Each experiements is sampled for 10 times and each sample contains at least 1 iteration. Then average and standard deviation is reported.

To ensure noninterference across multiple benchmarks, separate transactions/file loading are used in individual benchmarks.

3 Experiment Results

3.1 Retrieval Performance Comparison

3.1.1 Retrieval Test 1

Query:

```
SELECT amount FROM salary ORDER BY employee_id LIMIT [query_limit];
```

Results:

Engine \ Data Size	100	500	1000	5000	10000
PostgreSQL	1.247 91 ms	1.2535 ms	1.482 45 ms	2.966 59 ms	3.150 38 ms
openGauss	1.139 57 ms	1.232 23 ms	1.254 02 ms	2.185 57 ms	3.502 67 ms
JSON (w/o file loading)	14.3659 ns	123.442 ns	607.379 ns	6.362 33 μ s	26.198 μ s
JSON (w/ file loading)	742.861 ms	742.861 ms	742.861 ms	742.867 ms	742.887 ms

Engine \ Data Size	50000	100000	500000	1000000
PostgreSQL	15.0285 ms	20.8607 ms	97.3349 ms	189.96 ms
openGauss	14.1096 ms	27.3405 ms	617.438 ms	681.953 ms
JSON (w/o file loading)	686.888 μ s	12.0072 ms	3.023 01 ms	4.680 98 ms
JSON (w/ file loading)	743.548 ms	744.062 ms	745.884 ms	747.542 ms

Table 1: Retrieval Test 1 Benchmark Results

3.1.2 Retrieval Test 2

Query:

```
SELECT e.first_name, e.last_name, d.dept_name, t.title, s.amount
FROM employee e
JOIN department_employee de ON e.id = de.employee_id
JOIN department d ON de.department_id = d.id
JOIN title t ON e.id = t.employee_id
JOIN salary s ON e.id = s.employee_id
WHERE e.hire_date > [date];
```

Results:

Engine \ Filter	>1997-01-01	>1998-01-01	>1999-01-01	>2000-01-01
PostgreSQL	17.547 ms	34.5756 ms	94.0941 ms	184.327 ms
openGauss	34.0926 ms	182.512 ms	313.017 ms	467.861 ms
JSON (w/o file loading)	10.5896 ms	12.820 ms	15.0948 ms	16.6291 ms
JSON (w/ file loading)	831.721 ms	833.951 ms	847.226 ms	848.7601 ms

3.1.3 Retrieval Test 3

Query:

```
SELECT d.dept_name, COUNT(e.id), AVG(s.amount)
FROM department d
JOIN department_employee de ON d.id = de.department_id
JOIN employee e ON de.employee_id = e.id
JOIN salary s ON e.id = s.employee_id
WHERE s.to_date = '9999-01-01'
GROUP BY d.dept_name;
```

Results:

Engine	Time
PostgreSQL	270.564 ms
openGauss	615.559 ms
JSON (w/o file loading)	20.7729 ms
JSON (w/ file loading)	712.5289 ms

3.1.4 Retrieval Test 4

Query:

```
SELECT * FROM employee WHERE first_name LIKE 'John%' OR last_name LIKE '%son';
```

Results:

Engine	Time
PostgreSQL	21.875 ms
openGauss	42.291 ms
JSON (w/o file loading)	5.746 ms
JSON (w/ file loading)	169.184 ms

3.1.5 Retrieval Test 5

Query:

```
SELECT * FROM employee
WHERE hire_date BETWEEN [start_date] AND [end_date]
ORDER BY hire_date DESC;
```

Results:

Engine	Range		
	1995-01-01 to 1997-12-31	1992-01-01 to 2000-12-31	1990-01-01 to 2022-12-31
PostgreSQL	34.0679 ms	65.4671 ms	86.5669 ms
openGauss	48.0719 ms	92.2885 ms	127.505 ms
JSON (w/o file loading)	11.22 ms	12.5463 ms	11.9194 ms
JSON (w/ file loading)	149.229 ms	150.555 ms	149.9284 ms

3.2 Update Performance Comparison

3.2.1 Update Test 1

Update:

```
UPDATE employee SET first_name = 'John' WHERE id = [random_id];
```

Results:

Engine	Time
PostgreSQL	3.962 89 ms
openGauss	3.517 69 ms
JSON	264.204 ms

Table 2: Update Test 1 Performance

3.2.2 Update Test 2

Update:

```
UPDATE salary SET amount = amount + 1 WHERE to_date = '9999-01-01';
```

Results:

Engine	Time
PostgreSQL	838.78 ms
openGauss	1.495 78 s
JSON	1.400 19 s

Table 3: Update Test 2 Performance

Conclusion:

3.2.3 Update Test 3

Update:

```
BEGIN;  
UPDATE department_employee SET to_date = CURRENT_DATE  
  WHERE employee_id = [random_id] AND to_date = '9999-01-01';  
INSERT INTO department_employee VALUES ([random_id], [random_dept_id],  
  ↪ CURRENT_DATE, '9999-01-01');  
COMMIT;
```

Results:

Engine	Time
PostgreSQL	34.2222 ms
openGauss	55.9333 ms
JSON	149.989 ms

Table 4: Update Test 3 Performance

Conclusion:

3.2.4 Update Test 4

Update:

```
UPDATE title SET title = 'Senior Engineer'  
  WHERE title = 'Engineer' AND from_date < '2010-01-01';
```

Results:

Engine	Time
PostgreSQL	26.7132 ms
openGauss	46.4891 ms
JSON	218.601 ms

Table 5: Update Test 4 Performance

Conclusion:

3.3 Advanced Comparisons: Different File Format

For additional comparison, we used the **BEVE** (Binary Efficient Versatile Encoding). It's a tagged binary data specification designed for higher performance and scientific computing. For more infor, see the <https://github.com/beve-org/beve>. **Query:**

```
SELECT amount FROM salary ORDER BY employee_id LIMIT [query_limit];
```

Results:

Engine \ Data Size	100	500	1000	5000	10000
PostgreSQL	1.247 91 ms	1.2535 ms	1.482 45 ms	2.966 59 ms	3.150 38 ms
openGauss	1.139 57 ms	1.232 23 ms	1.254 02 ms	2.185 57 ms	3.502 67 ms
JSON (w/ file loading)	742.861 ms	742.861 ms	742.861 ms	742.867 ms	742.887 ms
BEVE (w/ file loading)	206.300 ms	206.300 ms	206.301 ms	206.303 ms	206.310 ms

Engine \ Data Size	50000	100000	500000	1000000
PostgreSQL	15.0285 ms	20.8607 ms	97.3349 ms	189.96 ms
openGauss	14.1096 ms	27.3405 ms	617.438 ms	681.953 ms
JSON (w/ file loading)	743.548 ms	744.062 ms	745.884 ms	747.542 ms
BEVE (w/ file loading)	206.509 ms	206.647 ms	208.035 ms	210.949 ms

Table 6: Additional Format Benchmark Results

4 Analysis and Discussion

1. DBMS vs. File Operations:

- For any applications that need complex data query and modifications, DBMS is way better than manual file operations. They eliminate the need to load the whole file into memory every operation, and reduce the develop burden as we no longer need to write ad-hoc code for specific tasks. They can also optimize our operations and maintain data integrity through transactions.
- For very specific circumstances where we only need to perform very simple tasks on already cached data, e.g. traversal, the file-based approach can be faster. This is mainly because it performs a direct linear scan without overhead of SQL parsing, optimizing, planning, and client-server communication. But these scenarios are considerably rare in real-world applications. We can already see from the results that the BEVE format can provide performance close to DBMSs.

2. PostgreSQL vs. openGauss:

Across almost every test, PostgreSQL demonstrated significantly better performance than openGauss. The performance gap was most pronounced in complex queries (Retrieval Test 2, 3) and large-scale updates (Update Test 2), where PostgreSQL was often twice as fast or more. We consider that this lead can be attributed to several factors: more mature optimizer, efficient execution engine, less overhead on personal computer (since openGauss is designed specifically for enterprises).

5 Conclusion

This project conducted a performance evaluation of two DBMSs, PostgreSQL and openGauss, against manual file operations using JSON and BEVE formats for an realistic employee database. The results lead to two conclusions:

- First, DBMS are superior to file-based operations for dynamic applications involving complex queries and data modifications. Although the manual file processing is faster if we have the whole file in memory, the extreme cost of file I/O renders this approach impractical since it will require much more memory and is inefficient for data updates (since our naive implementation don't have indexing and needs an linear search when finding the data).
- Second, in a direct comparison of the two DBMSs, PostgreSQL exhibited consistently better performance than openGauss across a wide range of benchmarks.

In summary, while optimized file formats like BEVE have a potential usage in high-performance scenarios, the use of DBMS is recommended for nearly all applications due to its performance, scalability, and maintainability.

A Appendix A: Source Code Repository

The complete source code and report source can be found at [nofel248/cs213-project1](https://github.com/nofel248/cs213-project1).

B Appendix B: Generative AI Usage

Deepseek-R1 was used to generate the structure of this project report and convert the raw benchmark results into LaTeX tables.

C Appendix C: Raw Experimental Data

C.1 Retrieval Test 1

Query	Samples	Iterations	Mean Time	Std Dev	Est. Run Time
LIMIT 100	10	1	1.247 91 ms	145.879 μ s	5.259 32 ms
LIMIT 500	10	1	1.2535 ms	149.054 μ s	5.461 28 ms
LIMIT 1000	10	1	1.482 45 ms	144.062 μ s	6.947 05 ms
LIMIT 5000	10	1	2.966 59 ms	480.329 μ s	16.088 ms
LIMIT 10000	10	1	3.150 38 ms	243.688 μ s	23.5395 ms
LIMIT 50000	10	1	15.0285 ms	7.278 59 ms	92.4291 ms
LIMIT 100000	10	1	20.8607 ms	617.642 μ s	190.795 ms
LIMIT 500000	10	1	97.3349 ms	1.452 21 ms	1.360 07 s
LIMIT 1000000	10	1	189.96 ms	2.820 49 ms	2.436 46 s

Table 7: PostgreSQL SELECT Performance (Retrieval Test 1)

Query	Samples	Iterations	Mean Time	Std Dev	Est. Run Time
LIMIT 100	10	1	1.139 57 ms	251.143 μ s	5.532 79 ms
LIMIT 500	10	1	1.232 23 ms	249.885 μ s	7.850 08 ms
LIMIT 1000	10	1	1.254 02 ms	128.532 μ s	7.835 51 ms
LIMIT 5000	10	1	2.185 57 ms	86.997 μ s	17.853 ms
LIMIT 10000	10	1	3.502 67 ms	162.915 μ s	31.3957 ms
LIMIT 50000	10	1	14.1096 ms	190.007 μ s	134.13 ms
LIMIT 100000	10	1	27.3405 ms	867.596 μ s	261.998 ms
LIMIT 500000	10	1	617.438 ms	32.8302 ms	5.919 87 s
LIMIT 1000000	10	1	681.953 ms	29.958 ms	7.031 45 s

Table 8: openGauss SELECT Performance (Retrieval Test 1)

Query	Samples	Iterations	Mean Time	Std Dev	Est. Run Time
File Loading	10	1	742.861 ms	96.3481 ms	6.406 74 s
Reading 100	10	1667	14.3659 ns	0.680 831 ns	216.71 μ s
Reading 500	10	226	123.442 ns	11.9236 ns	223.74 μ s
Reading 1000	10	58	607.379 ns	100.921 ns	225.62 μ s
Reading 5000	10	12	6.362 33 μ s	2.714 59 μ s	241.44 μ s
Reading 10000	10	6	26.198 μ s	9.676 07 μ s	253.02 μ s
Reading 50000	10	1	686.888 μ s	285.85 μ s	381.31 μ s
Reading 100000	10	1	1.200 72 ms	432.117 μ s	2.844 76 ms
Reading 500000	10	1	3.023 01 ms	939.154 μ s	19.0127 ms
Reading 1000000	10	1	4.680 98 ms	2.554 51 ms	61.3399 ms

Table 9: JSON Retrieval Test 1 Results

Query	Samples	Iterations	Mean Time	Std Dev	Est. Run Time
File Loading	10	1	206.3 ms	10.0127 ms	2.043 34 s
Reading 100	10	1456	17.636 ns	3.114 38 ns	218.4 μ s
Reading 500	10	203	115.507 ns	17.2625 ns	221.27 μ s
Reading 1000	10	48	515.583 ns	46.9934 ns	225.6 μ s
Reading 5000	10	11	3.302 55 μ s	137.585 ns	233.86 μ s
Reading 10000	10	6	10.4497 μ s	1.632 65 μ s	254.94 μ s
Reading 50000	10	1	208.638 μ s	104.035 μ s	348.13 μ s
Reading 100000	10	1	346.648 μ s	45.1029 μ s	641.83 μ s
Reading 500000	10	1	1.734 75 ms	469.347 μ s	18.2551 ms
Reading 1000000	10	1	4.6492 ms	1.907 27 ms	36.3955 ms

Table 10: BEVE Retrieval Test 1 Results

C.2 Retrieval Test 2

Join Filter	Samples	Iterations	Mean Time	Std Dev	Est. Run Time
> 2000-01-01	10	1	17.547 ms	824.177 μ s	126.665 ms
> 1999-01-01	10	1	34.5756 ms	2.807 65 ms	4.122 39 s
> 1998-01-01	10	1	93.0941 ms	49.3798 ms	7.892 15 s
> 1997-01-01	10	1	184.327 ms	31.322 ms	6.835 17 s

Table 11: PostgreSQL Join Query Performance (Retrieval Test 2)

Join Filter	Samples	Iterations	Mean Time	Std Dev	Est. Run Time
> 2000-01-01	10	1	34.0926 ms	934.005 μ s	368.575 ms
> 1999-01-01	10	1	182.512 ms	5.993 36 ms	1.913 13 s
> 1998-01-01	10	1	313.017 ms	11.014 ms	3.154 03 s
> 1997-01-01	10	1	467.861 ms	6.567 89 ms	4.699 77 s

Table 12: openGauss Join Query Performance (Retrieval Test 2)

Join Filter	Samples	Iterations	Mean Time	Std Dev	Est. Run Time
JSON Reading	10	1	832.131 ms	14.9833 ms	8.547 48 s
> 2000-01-01	10	1	10.5896 ms	2.255 61 ms	83.7742 ms
> 1999-01-01	10	1	12.82 ms	1.945 56 ms	110.761 ms
> 1998-01-01	10	1	15.0948 ms	2.0567 ms	133.673 ms
> 1997-01-01	10	1	16.6291 ms	2.138 56 ms	159.399 ms

Table 13: JSON Join Query Performance (Retrieval Test 2)

C.3 Retrieval Test 3

Aggregate	Samples	Iterations	Mean Time	Std Dev	Est. Run Time
Aggregate	10	1	270.564 ms	18.8131 ms	5.108 55 s

Table 14: PostgreSQL Aggregate Query Performance (Retrieval Test 3)

Aggregate	Samples	Iterations	Mean Time	Std Dev	Est. Run Time
Aggregate	10	1	615.559 ms	73.6808 ms	6.646 75 s

Table 15: openGauss Aggregate Query Performance (Retrieval Test 3)

Aggregate	Samples	Iterations	Mean Time	Std Dev	Est. Run Time
JSON Reading	10	1	691.756 ms	70.7634 ms	5.145 14 s
Aggregate	10	1	20.7729 ms	3.315 91 ms	156.136 ms

Table 16: JSON Aggregate Query Performance (Retrieval Test 3)

C.4 Retrieval Test 4

Engine	Samples	Iterations	Mean Time	Std Dev	Est. Run Time
PostgreSQL	10	1	21.8753 ms	1.605 ms	197.325 ms
openGauss	10	1	42.2907 ms	677.885 μ s	426.83 ms
JSON	10	1	5.745 89 ms	1.892 58 ms	52.3803 ms
JSON File Loading	10	1	163.438 ms	6.898 17 ms	1.772 27 s

Table 17: Text Search Performance (Retrieval Test 4)

C.5 Retrieval Test 5

Date Range	Samples	Iterations	Mean Time	Std Dev	Est. Run Time
1995-01-01 to 1997-12-31	10	1	34.0679 ms	2.954 71 ms	283.784 ms
1992-01-01 to 2000-12-31	10	1	65.4671 ms	2.147 29 ms	655.673 ms
1990-01-01 to 2022-12-31	10	1	86.5669 ms	3.910 15 ms	845.364 ms

Table 18: PostgreSQL Date Range Query Performance (Retrieval Test 5)

Date Range	Samples	Iterations	Mean Time	Std Dev	Est. Run Time
1995-01-01 to 1997-12-31	10	1	48.0719 ms	2.387 09 ms	470.786 ms
1992-01-01 to 2000-12-31	10	1	92.2885 ms	4.682 16 ms	922.416 ms
1990-01-01 to 2022-12-31	10	1	127.505 ms	5.400 44 ms	1.243 96 s

Table 19: openGauss Date Range Query Performance (Retrieval Test 5)

Date Range	Samples	Iterations	Mean Time	Std Dev	Est. Run Time
JSON Employee Loading	10	1	138.009 ms	9.538 02 ms	1.405 78 s
1995-01-01 to 1997-12-31	10	1	11.22 ms	2.585 05 ms	106.138 ms
1992-01-01 to 2000-12-31	10	1	12.5463 ms	2.913 84 ms	118.255 ms
1990-01-01 to 2022-12-31	10	1	11.9194 ms	1.850 84 ms	140.438 ms

Table 20: JSON Date Range Query Performance (Retrieval Test 5)

C.6 Update Test 1

Update 1	Samples	Iterations	Mean Time	Std Dev	Est. Run Time
PostgreSQL	10	1	3.962 89 ms	1.131 43 ms	26.895 ms
openGauss	10	1	3.517 69 ms	795.448 μ s	27.6951 ms
JSON	10	1	264.204 ms	13.6148 ms	2.606 46 s

Table 21: Update Test 1 Performance

C.7 Update Test 2

Update 2	Samples	Iterations	Mean Time	Std Dev	Est. Run Time
PostgreSQL	10	1	838.78 ms	112.592 ms	14.2564 s
openGauss	10	1	1.495 78 s	127.423 ms	20.9112 s
JSON	10	1	1.400 19 s	142.632 ms	15.2397 s

Table 22: Update Test 2 Performance

C.8 Update Test 3

Update 3	Samples	Iterations	Mean Time	Std Dev	Est. Run Time
PostgreSQL	10	1	34.2222 ms	887.247 μ s	310.753 ms
openGauss	10	1	55.9333 ms	6.256 15 ms	526.343 ms
JSON	10	1	149.989 ms	4.710 99 ms	1.549 74 s

Table 23: Update Test 3 Performance

C.9 Update Test 4

Update 4	Samples	Iterations	Mean Time	Std Dev	Est. Run Time
PostgreSQL	10	1	26.7132 ms	5.256 92 ms	4.9482 s
openGauss	10	1	46.4891 ms	6.590 66 ms	8.449 97 s
JSON	10	1	218.601 ms	9.426 36 ms	2.274 s

Table 24: Update Test 4 Performance

D Appendix D: Test Code

D.1 Retrieval Test 1

```
#define SALARY_BENCH_PG(LIM)
↪ \
BENCHMARK_ADVANCED("PostgreSQL salary ORDER BY employee_id LIMIT "
↪ #LIM)(Catch::Benchmark::Chronometer meter) {
↪     \
    pqxx::connection conn{pg_conn_str.data()};
    ↪ \
    pqxx::read_transaction tx{conn};
    ↪ \
    tx.exec("SET search_path TO employees");
    ↪ \
    meter.measure([&] {
    ↪     \
        std::int64_t sum = 0;
        ↪ \
        for (auto [amount]:
        ↪     \
            tx.query<std::int64_t>("SELECT amount FROM salary ORDER BY
            ↪ employee_id LIMIT " #LIM)) {
            ↪ \
                sum += amount;
            ↪ \
        }
        ↪ \
        return sum;
    ↪ \
}
```

```

    });
    ↪ \
}

TEST_CASE("retrieval-test-1-postgresql",
    ↪ "[retrieval-post][retrieval-test-1][retrieval-test]") {
    try {
        SALARY_BENCH_PG(100);
        SALARY_BENCH_PG(500);
        SALARY_BENCH_PG(1000);
        SALARY_BENCH_PG(5000);
        SALARY_BENCH_PG(10000);
        SALARY_BENCH_PG(50000);
        SALARY_BENCH_PG(100000);
        SALARY_BENCH_PG(500000);
        SALARY_BENCH_PG(1000000);
    } catch (pqxx::sql_error const &e) {
        FAIL(std::format("Query failed with: {}", e.what()));
    } catch (std::exception const &e) {
        FAIL(std::format("Failed to connect to PostgreSQL server '{}'",
            ↪ e.what()));
    }
}

#define SALARY_BENCH_OG(LIM)
    ↪ \
    BENCHMARK_ADVANCED("openGauss salary ORDER BY employee_id LIMIT "
    ↪ #LIM)(Catch::Benchmark::Chronometer meter) {
        ↪ \
        pqxx::connection conn{og_conn_str.data()};
        ↪ \
        pqxx::read_transaction tx{conn};
        ↪ \
        tx.exec("SET search_path TO employees");
        ↪ \
        meter.measure([&] {
            ↪ \
            std::int64_t sum = 0;
            ↪ \
            for (auto [amount]:
                ↪ \
                tx.query<std::int64_t>("SELECT amount FROM salary ORDER BY
                ↪ employee_id LIMIT " #LIM)) {
                ↪ \
                sum += amount;
                ↪ \
            }
            ↪ \
            return sum;
            ↪ \
        });
        ↪ \
    }

TEST_CASE("retrieval-test-1-opengauss",
    ↪ "[retrieval-open][retrieval-test-1][retrieval-test]") {
    try {
        SALARY_BENCH_OG(100);
        SALARY_BENCH_OG(500);
        SALARY_BENCH_OG(1000);
        SALARY_BENCH_OG(5000);
        SALARY_BENCH_OG(10000);
        SALARY_BENCH_OG(50000);
    }
}

```

```

        SALARY_BENCH_OG(100000);
        SALARY_BENCH_OG(500000);
        SALARY_BENCH_OG(1000000);
    } catch (pqxx::sql_error const &e) {
        FAIL(std::format("Query failed with: {}", e.what()));
    } catch (std::exception const &e) {
        FAIL(std::format("Failed to connect to PostgreSQL server '{}'",
            ↪ e.what()));
    }
}

#define JSON_READ_BENCH(N)
↪ \
BENCHMARK_ADVANCED("JSON Reading " #N)(Catch::Benchmark::Chronometer meter) {
    ↪ \
        std::vector<Salary> salaries;
        ↪ \
        if (auto const error = glz::read_file_json(salaries,
            ↪ "../employees/salary.json", std::string{})) {
            ↪ FAIL(std::format("Failed to read JSON file: {}",
                ↪ glz::format_error(error)));
        }
        ↪ \
        meter.measure([&] {
            ↪ \
                std::int64_t sum = 0;
                ↪ \
                for (auto const &s: salaries | std::views::take(N))
                    ↪ \
                        sum += s.amount;
                ↪ \
                return sum;
            ↪ \
        });
    }

TEST_CASE("retrieval-test-1-json",
    ↪ "[retrieval-json][retrieval-test-1][retrieval-test]") {
    BENCHMARK_ADVANCED("JSON File Loading")(Catch::Benchmark::Chronometer meter)
    ↪ {
        meter.measure([&] {
            std::vector<Salary> salaries;
            if (auto const error = glz::read_file_json(salaries,
                ↪ "../employees/salary.json", std::string{}))
                ↪ FAIL(std::format("Failed to read JSON file: {}",
                    ↪ glz::format_error(error)));
            ↪ return salaries.size();
        });
    };

    JSON_READ_BENCH(100);
    JSON_READ_BENCH(500);
    JSON_READ_BENCH(1000);
    JSON_READ_BENCH(5000);
    JSON_READ_BENCH(10000);
    JSON_READ_BENCH(50000);
    JSON_READ_BENCH(100000);
    JSON_READ_BENCH(500000);
    JSON_READ_BENCH(1000000);
}

```

```

#define BEVE_READ_BENCH(N)
↳ \
    BENCHMARK_ADVANCED("BEVE Reading " #N)(Catch::Benchmark::Chronometer meter) {
↳ \
        std::vector<Salary> salaries;
        ↳ \
        if (auto const error = glz::read_file_beve(salaries,
↳ \
            "../employees/salary.beve", std::string{})) {
            FAIL(std::format("Failed to read BEVE file: {}",
↳ \
                glz::format_error(error)));
        }
        ↳ \
        meter.measure([&] {
        ↳ \
            std::int64_t sum = 0;
            ↳ \
            for (auto const &s: salaries | std::views::take(N))
            ↳ \
                sum += s.amount;
            ↳ \
            return sum;
            ↳ \
        });
        ↳ \
    }

TEST_CASE("retrieval-test-1-beve",
↳ \
    "[retrieval-beve][retrieval-test-1][retrieval-test]") {
    BENCHMARK_ADVANCED("BEVE File Loading")(Catch::Benchmark::Chronometer meter)
    ↳ {
        meter.measure([&] {
            std::vector<Salary> salaries;
            if (auto const error = glz::read_file_beve(salaries,
↳ \
                "../employees/salary.beve", std::string{}))
                FAIL(std::format("Failed to read BEBE file: {}",
↳ \
                    glz::format_error(error)));
            return salaries.size();
        });
    };

    BEVE_READ_BENCH(100);
    BEVE_READ_BENCH(500);
    BEVE_READ_BENCH(1000);
    BEVE_READ_BENCH(5000);
    BEVE_READ_BENCH(10000);
    BEVE_READ_BENCH(50000);
    BEVE_READ_BENCH(100000);
    BEVE_READ_BENCH(500000);
    BEVE_READ_BENCH(1000000);
}

```